ShanghaiTech University

CS101 Algorithms and Data Structures
Fall 2024

Homework 9

Wenye Xiong 2023533141

Due date: December 4, 2024, at 23:59

1. Please write your solutions in English.
2. Submit your solutions to Gradescope.
3. Set your FULL name to your Chinese name and your STUDENT ID correctly in Gradescope account settings.
4. If you want to submit a handwritten version, scan it clearly. `CamScanner` is recommended.
5. We recommend you to write in LaTeX.
6. When submitting, match your solutions to the problems correctly.
7. No late submission will be accepted.
8. Violations to any of the above may result in zero points.

1. (8 points) Multiple Choices

Each question has **one or more** correct answer(s). Select all the correct answer(s). For each question, you will get 0 points if you select one or more wrong answers, but you will get 1 point if you select a non-empty subset of the correct answers.

Write your answers in the following table.

(a)	(b)	(c)	(d)
BC	ABD	BC	D

(a) (2') Which of the following statements about topological sort is/are true?

A. A sub-graph of a DAG may not have a topological sorting.

B. Any directed tree has a topological sorting.

C. It's possible for a DAG to have multiple topological sortings.

D. Since we have to scan all vertices to find those with zero in-degree in each iteration, the run time of topological sort is $\Omega(|V|^2)$.

(b) (2') Which of the following statements about topological sort is/are true?

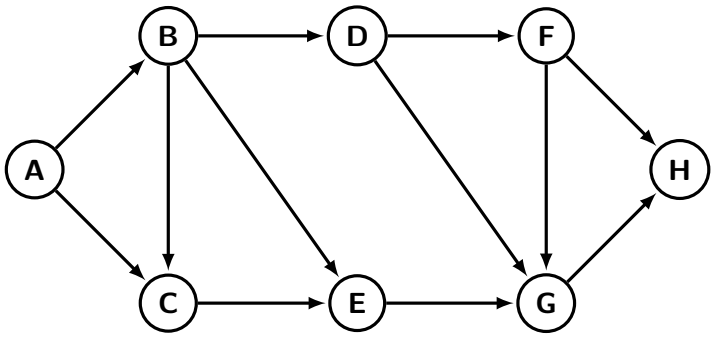
A. For a connected graph, we can determine if it has a cycle in $\Theta(|E|)$ time.

B. A critical path in a DAG is a path from the source to the sink with the maximum total weights.

C. A DAG with all different weighted edges has one unique critical path.

D. A DAG has at least one source and at least one sink.

(c) (2') Which of the following sequences is/are **Topological Sorting**(s) of the given DAG?



A. A B C E G D F H

B. A B C D E F G H

C. A B D C F E G H

D. A B D E C F G H

(d) (2') For the coin-changing problem, denote

- $C = (c_1, c_2, \cdots, c_k)$: the denominations of coins, where $1 = c_1 < c_2 < \cdots < c_k$;

- X_n^*: an optimal solution, i.e., a multi-set of coins which has the minimum number of coins to make change for n;
- X_n: the greedy solution, solved by

$$X_n = \begin{cases} \{\}, & n = 0 \\ \{c_t\} \cup X_{n-c_t}, & n \geq 1, \text{where } c_t = \max_{c_i \leq n} c_i \text{ is the largest coin that can be used} \end{cases}$$

For example, if $C = (1, 2, 3)$, then X_4^* can be either $\{2, 2\}$ or $\{3, 1\}$, and $X_4 = \{3, 1\}$.

Which of the following statements is/are true?

A. If $\forall i \in [3, n], c_i = c_{i-1} + c_{i-2}$, then $\forall n, |X_n^*| = |X_n|$.

B. $\exists C, \exists i$ with $2c_i > c_{i+1}$, such that $\forall n, |X_n^*| = |X_n|$.

C. If $\forall i, 2c_i \leq c_{i+1}$, then $\forall n, |X_n^*| = |X_n|$.

D. If $\forall i \in [2, n], \frac{c_i}{c_{i-1}}$ is an integer, then $\forall n, |X_n^*| = |X_n|$.

2. (10 points) Test Paper Arrangement Task

Midterm week is finally over! Now the CS101 course staff is filing the test papers. Suppose there are t test papers waited to be filed, and the course staff was assigned n file cabinets with various capacities. Mark the capacity of the file cabinet as c_i $(i = 1, 2, \cdots, n)$. It's guaranteed that $\sum_{i=1}^{n} c_i \geq t$.

(a) (5') Given the number of test papers t and the capacities of each of the n cabinets $c_1, c_2, \cdots, c_n$, come up with a greedy algorithm that can file the papers in cabinets such that the least number of cabinets is used. You should give out the steps of your algorithm **(as efficient as possible)** and the time complexity (tight).

Solution:

Algorithm 1 Greedy Algorithm for Test Paper Arrangement

```
Sort the capacities of the cabinets in descending order: c_1 > c_2 > ··· > c_n.
Initialize the number of test papers left to be filed: t.
Initialize the number of cabinets used: m = 0.
for i = 1 to n do
    t = t − c_i
    if t <= 0 then
        m = i
        break
    end if
end for
return m
```

Time Complexity: $O(n)$

(b) (5') Prove the optimality of your algorithm using **Exchange Arguments** (any optimal solution can be iteratively exchanged to your greedy solution without making it worse). Answer the questions in step 4.5. to give your sufficient proof.

Proof:

1. Assume $c_1 > c_2 > \cdots > c_n$ after sorted. Denote a solution $X = (x_1, x_2, \cdots, x_n)$ where we put x_i test papers in the cabinet of capacity c_i. And the measurement $m(X)$ is counting how many cabinets have at least one test paper.
2. Assume the optimal solution is X^*, and the greedy solution is X.
3. Consider the first point of discrepancy between X^* and X at index i (i.e. $x_i^* \neq x_i$).
4. How to perform changes to make $x_i^* = x_i$? You should ensure that such changes is always valid in all possible scenarios of X^*.

   Solution: If x_i^* is greater than zero, which means it is used in X^*, we can randomly choose $x_i - x_i^*$ test papers from the rest of X^*, which is the set of x_{i+1}^* to x_n^*. Then we put these test papers into the cabinet of capacity c_i. In that way, we can make $x_i^* = x_i$. This operation is always valid because sum of X and X^* are equal, so there are at least $x_i - x_i^*$ test papers in the rest of X^*.
   And for $x_i^* = 0$, we can go on to find a j such that $x_j^* > 0$ and $j > i$, then we can move x_j^* test papers to the cabinet of capacity c_i. This operation is also valid because we sort the cabinets in descending order, so for $j > i$, $x_j^* \leq c_j < c_i$, which means the cabinet of capacity c_i can hold the test papers from cabinet j.

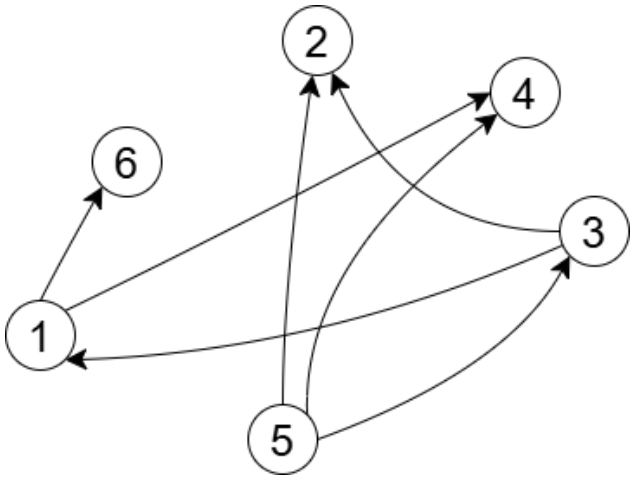
5. Why such changes doesn't make X^* worse? You should consider all possible scenarios of X^*.

   Solution: For situation where x_i^* is greater than zero, the number of cabinets used in X^* will not increase. Because we only move test papers from the rest of X^* to a used cabinet of capacity c_i. That will do no contribution to the number of cabinets used.
   For situation where $x_i^* = 0$, the number of cabinets used in X^* will not increase. Even though we do let the cabinet of capacity c_i hold some test papers, we also move all test papers from the cabinet of capacity c_j. So in general we are just exchanging the cabinets used in X^*, which will not increase the number of cabinets used.
   Thus we are sure to say that such changes doesn't make X^* worse.

6. Continue Applying steps 3.4.5. until $X^* = X$. Finally we prove that X is an equally optimal solution.

3. (10 points) Topological Sorting

Consider the unweighted directed graph G below.



(a) (5') How many topological sort results are there for the graph? State all possible results and show your calculated process.

Solution: There are 8 topological sort results for the graph. They are as follows:

5 3 1 2 4 6
5 3 1 2 6 4
5 3 1 4 2 6
5 3 1 4 6 2
5 3 1 6 2 4
5 3 1 6 4 2
5 3 2 1 4 6
5 3 2 1 6 4

Process:

1. Find the vertices with zero in-degree.
2. Remove the vertices with zero in-degree and their outgoing edges, then update the in-degree of the remaining vertices
3. Repeat step 1 and 2 until all vertices are removed

We can find that at first, only 5 has zero in-degree. So we can only remove 5 and its outgoing edges. Then we can find that only 3 has zero in-degree. So we can only remove 3 and its outgoing edges. This gives us the first two vertices in the topological sort: 5, 3
Then, after removing 5 and 3, we can find that both 1 and 2 have zero in-degree. Now we have our first diverge. We first talk about the case where we choose 1 first.

After removing 1 and its outgoing edges, we can find that all remaining vertices, 2 4 6, have zero in-degree. So we can remove them in any order. This gives us 6 types of topological sort: 5 3 1 2 4 6, 5 3 1 2 6 4, 5 3 1 4 2 6, 5 3 1 4 6 2, 5 3 1 6 2 4, 5 3 1 6 4 2.
Then we talk about the case where we choose 2 first. After removing 2 and its outgoing edges, we can find that still only vertex 1 has zero in-degree. So we can remove 1 and its outgoing edges. After that both 4 and 6 have zero in-degree. So we can remove them in any order. This gives us 2 more types of topological sort: 5 3 2 1 4 6, 5 3 2 1 6 4.
Thus we have 8 topological sort results for the graph.

(b) (5') Construct an example of a single directed edge that, when added to G, would make it impossible to create a valid topological ordering of the graph. Then, determine the total number of such edges that could be added to G and explain why each of these edges prevents a topological ordering.
(Ensure that this edge is not a self-loop and does not duplicate an existing edge in G.)

Solution: Example: add an edge from 6 to 5. This will make it impossible to create a valid topological ordering of the graph because at first step there is no vertex with zero in-degree.
To find all the edges that could be added to G to make it impossible to create a valid topological ordering, we can find all the edges such that it will make the graph has no vertex with zero in-degree at some point.
For the first step, if we add an edge from an arbitrary vertex to 5, then there will be no vertex with zero in-degree. This will contribute to 5 edges that could be added to G to make it impossible to create a valid topological ordering.
Next, consider vertex 3. If we add an edge from an arbitrary vertex except 5 to 3, then there will be no vertex with zero in-degree after removing 5. This will contribute to 4 edges that could be added to G to make it impossible to create a valid topological ordering.

Next, we talk about the subgraph of 1 2 4 6. It is easy to see that in this subgraph, the only way to make it impossible to create a valid topological ordering is to add an edge that is a reverse of an existing edge(that is to say, add an edge from 4 to 1 or from 6 to 1). This will contribute to 2 edges that could be added to G to make it impossible to create a valid topological ordering.
Thus in total, there are 5+4+2=11 edges that could be added to G to make it impossible to create a valid topological ordering.

4. (8 points) Greedy algorithm

You are given a set of n cakes. Each cake i has:

- A **satisfaction value** $s_i \in \mathbb{N}^+$, representing the enjoyment you get from eating the cake.
- An **expiration date** $e_i \in \mathbb{N}^+$, specifying the latest day (starting from day 1) the cake can be eaten. If not eaten by day e_i, the cake is no longer valid.

You can eat at most **one cake per day**, starting from day 1, up to day $E = \max_{i \in [1,n]} \{e_i\}$ i.e. the maximum expiration date of any cake. Your goal is to determine the best strategy to eat the cakes to **maximize the total satisfaction**, defined as the sum of the satisfaction values of the cakes you consume.

To illustrate, consider the following example of $n = 9$ cakes:

i	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
s_i	10	20	10	15	14	40	18	5	20
e_i	5	3	3	3	5	4	5	7	5

The maximum expiration date is $m = 7$, so you need to decide which cake to eat on each day from day 1 to day 7. Note that:

- You can only eat cake i on day j if $j \leq e_i$, due to the expiration date constraint.
- You can leave some days without eating any cake if that results in a higher total satisfaction.

We represent a possible allocation as $C = \{c_1, c_2, \ldots, c_E\}$, where c_j is the index of the cake eaten on day j, and "-" indicates no cake is eaten on that day.

Here are three valid allocations for this example:

- $\{1, 2, 3, 5, 7, 8, -\}$, with a total satisfaction of $77 = 10 + 20 + 10 + 14 + 18 + 5$.
- $\{1, 2, 3, 6, 7, -, 8\}$, with a total satisfaction of $103 = 10 + 20 + 10 + 40 + 18 + 5$.
- $\{2, 4, 6, 7, 9, -, 8\}$, with a total satisfaction of $118 = 20 + 15 + 40 + 18 + 20 + 5$.

For this instance, one of the optimal allocations is $\{2, 4, 6, 7, 9, -, 8\}$, resulting in the maximum total satisfaction of 118.

Notes for how to answer:

For the following two greedy algorithms, you need to judge whether it always find an optimal solution.

If not always, please give a counterexample, including the input, the greedy solution and an optimal solution.

If always, please give a strict proof by **"Greedy Stays Ahead" Arguments** including:

- (1pt) **A proper definition of a series of possible sub-problem solutions** $X_1, X_2, X_3, \cdots$, based on what this algorithm produces after each iteration.

  For example, in coin changing problem, sub-problem solution X_i is a multi-set of coins whose denominations add up to i.

- (1pt) **The definition of optimality**. You should give a series of measurements on each sub-problem $m_1(X_1), m_2(X_2), \cdots$, and then define the optimal sub-problem solution X_i^* by:

  $$X_i^* \in \arg\max\{m_i(X_i)\} \text{ or } X_i^* \in \arg\min\{m_i(X_i)\}$$

  For example, in coin changing problem, $m_i(X_i) = |X_i|$ is the number of coins, and the optimal solution is the minimal one.

- (2pts) **The proof that your greedy algorithm always stays ahead by induction**. Denote X_i as the greedy sub-problem solution, and you can prove like:

  $$m_{i-1}(X_{i-1}) = m_{i-1}(X_{i-1}^*) \implies m_i(X_i) = m_i(X_i^*)$$

  And usually it is proved by contradiction. For example, prove

  $$m_{i-1}(X_{i-1}) \neq m_{i-1}(X_{i-1}^*) \impliedby m_i(X_i) \neq m_i(X_i^*)$$

(a) (4') Consider days in ascending order, and choose the cake with maximal s_i first.

Algorithm 2 Days ascending, Cake maximal s_i

Require: A set of cakes, where each cake i has satisfaction s_i and expiration e_i.

Ensure: An allocation with the maximum total satisfaction.

```
C ← {−, −, ··· , −}                          ▷ Initial arrangement is empty
Sort the cakes by s_i in descending order
i ← 1                                        ▷ Consider cakes by s_i in descending order
for j = 1 to E do                            ▷ Consider days in ascending order
    while i ≤ n and e_i < j do
        i ← i + 1                            ▷ Skip expired cakes
    end while
    if i ≤ n then
        C_j ← i
    end if
end for
return C
```

Solution: This algorithm does not always find an optimal solution. A counterexample is as follows:

i	1	2	3
s_i	10	99	100
e_i	5	1	10

In this counterexample, the greedy algorithm will choose cake 3 on day 1, and then it can only choose cake 1 on day 2. The total satisfaction is 110. However, the optimal solution is to choose cake 2 on day 1 and cake 2 on day 2, cake 3 on day 3. The total satisfaction is 209.

(b) (4') Consider days in descending order, and choose the cake with maximal s_i first.

Algorithm 3 Days descending, Cake maximal s_i

Require: A set of cakes, where each cake i has satisfaction s_i and expiration e_i.

Ensure: An allocation with the maximum total satisfaction.

```
C ← {−, −, ··· , −}                          ▷ Initial arrangement is empty
H ← an empty heap, where H.top() returns the index i of the cake with maximal s_i.
for j = E downto 1 do                        ▷ Consider days in descending order
    for every i such that e_i = j do
        H.push(i)                            ▷ Cake i can be eaten before day j
    end for
    if H.notEmpty() then
        C_j ← H.top()                        ▷ Consider cakes by s_i in ascending order
        H.pop()
    end if
end for
return C
```

Solution: This algorithm always finds an optimal solution. We prove it by **"Greedy Stays Ahead" Arguments**:

- **A series of possible sub-problem solutions**:

  $$X_j = \{c_1, c_2, \cdots, c_j\}$$

  where c_j is the index of the cake eaten on day j.

- **The definition of optimality**:

  $$m_j(X_j) = \sum_{k=1}^{j} s_{c_k}$$

  and the optimal sub-problem solution is:

  $$X_j^* = \arg\max\{m_j(X_j)\}$$

- **The proof that your greedy algorithm always stays ahead by induction**:
  - **Base case**: X_1 is the greedy solution, and X_1^* is the optimal solution. Obviously, $m_1(X_1) = m_1(X_1^*)$.
  - **Inductive step**: Assume $m_{j-1}(X_{j-1}) = m_{j-1}(X_{j-1}^*)$. If $m_j(X_j) \neq m_j(X_j^*)$, then there must be a cake c_j^* such that $s_{c_j} < s_{c_j^*}$, where c_j is the cake in X_j. Since c_j is chosen by the greedy algorithm, for all cakes c_k with $e_{c_k} \geq j$, we have $s_{c_k} \leq s_{c_j}$. Therefore, $s_{c_j^*} \leq s_{c_j}$, which contradicts the assumption. Thus, $m_j(X_j) = m_j(X_j^*)$.

– By induction, the greedy algorithm always stays ahead. So we have proved that this algorithm always finds an optimal solution.